

S'y retrouver dans le dédale du code

Le principe « Closed-by-Default »















« Le principe « Closed-by-Default » élimine les impasses de votre labyrinthe mental et révèle un chemin plus direct vers la compréhension du code.


```
1 <?php
2
3 class Dog
4 {
5     const SOUND = 'Woof';
6
7     public $name;
8
9     public $microchipId;
10
11     public function __construct($name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     public function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch($item)
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
51 }
```

```
52 }
```

```
53         return $item;
```



```

1 <?php
2
3 class Dog
4 {
5     const SOUND = 'Woof';
6
7     public $name;
8
9     public $microchipId;
10
11     public function __construct($name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     public function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch($item)
23     {
24         $this->bark();
25         return $item;
26     }
27 }

```

Dog pourrait avoir des classes enfants

SOUND pourrait être de n'importe quel type

SOUND pourrait être lue depuis l'extérieur

\$name et **\$microchipId** pourraient être de n'importe quel type

\$name et **\$microchipId** pourraient être lues depuis l'extérieur

\$name et **\$microchipId** pourraient changer pendant l'exécution

\$name et **\$microchipId** pourraient être modifiées depuis l'extérieur

\$name et **\$microchipId** acceptent n'importe quel type dans le constructeur

bark et **fetch** pourraient être appelées depuis l'extérieur

bark et **fetch** pourraient retourner n'importe quel type

\$item accepte n'importe quel type dans la signature de **fetch**

PHP 5.0

Mots-clés **public** / **protected** / **private**

Classes finales (**final**)

Typage par classes et interfaces

PHP 5.0

```
1 <?php
2
3 class Dog
4 {
5     const SOUND = 'Woof';
6
7     public $name;
8
9     public $microchipId;
10
11     public function __construct($name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     public function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch($item)
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
51 }
52 }
53 return $item;
```

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct($name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item)
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
51 }
52 }
53 return $item;
```


PHP 5.0

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct($name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item)
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
28 }
29 }
30 return $item;
```

~~Dog~~ pourrait avoir des classes enfants

SOUND pourrait être de n'importe quel type

SOUND pourrait être lue depuis l'extérieur

\$name et **\$microchipId** pourraient être de n'importe quel type

~~\$name~~ et ~~\$microchipId~~ pourraient être lues depuis l'extérieur

\$name et **\$microchipId** pourraient changer pendant l'exécution

~~\$name~~ et ~~\$microchipId~~ pourraient être modifiées depuis l'extérieur

\$name et **\$microchipId** acceptent n'importe quel type dans le constructeur

~~bark~~ pourrait être appelée depuis l'extérieur

fetch pourrait être appelée depuis l'extérieur

bark et **fetch** pourraient retourner n'importe quel type

~~\$item~~ accepte n'importe quel type dans la signature de ~~fetch~~

PHP 7.0

Déclarations de types scalaires et de types de retour

PHP 7.0

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct($name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item)
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
51 }
52 }
53 return $item;
54 $this->bark();
```

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct(string $name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item): Ball
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
51 }
52 }
53 return $item;
54 $this->bark();
```


PHP 7.0

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct(string $name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item): Ball
23     {
24         $this->bark();
25         return $item;
26     }
27 }
28
29 }
30
31 return $item;
32 $this->bark();
```

SOUND pourrait être de n'importe quel type

SOUND pourrait être lue depuis l'extérieur

\$name pourrait être de n'importe quel type ?

\$microchipId pourrait être de n'importe quel type

\$name et **\$microchipId** pourraient changer pendant l'exécution

\$name accepte n'importe quel type dans le constructeur ?

\$microchipId accepte n'importe quel type dans le constructeur

fetch pourrait être appelée depuis l'extérieur

bark pourrait retourner n'importe quel type

~~**fetch** pourrait retourner n'importe quel type~~

PHP 7.0

Déclarations de types scalaires et de types de retour

Directive **declare(strict_types=1)**

PHP 7.0

```
1 <?php
2
3 final class Dog
4 {
5     const SOUND = 'Woof';
6
7     private $name;
8
9     private $microchipId;
10
11     public function __construct(string $name, $microchipId)
12     {
13         $this->name = $name;
14         $this->microchipId = $microchipId;
15     }
16
17     private function bark()
18     {
19         echo self::SOUND;
20     }
21
22     public function fetch(Ball $item): Ball
23     {
24         $this->bark();
25         return $item;
26     }
27 }
```

```
51 }
52 }
53
54 let $item = new Ball(1, 'Woof');
55 $item->bark();
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark()
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
51 }
52 }
53
54 let $item = new Ball(1, 'Woof');
55 $item->bark();
```


PHP 7.0

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark()
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
30
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark()
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
30
31 $dog = new Dog(123, 456);
```

```
31 $dog = new Dog(123, 456);
```


PHP 7.0

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark()
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

SOUND pourrait être de n'importe quel type

SOUND pourrait être lue depuis l'extérieur

\$name pourrait être de n'importe quel type

\$microchipId pourrait être de n'importe quel type

\$name et **\$microchipId** pourraient changer pendant l'exécution

~~**\$name** accepte n'importe quel type dans le constructeur~~

\$microchipId accepte n'importe quel type dans le constructeur

fetch pourrait être appelée depuis l'extérieur

bark pourrait retourner n'importe quel type

~~**fetch** pourrait retourner n'importe quel type~~

PHP 7.1

Visibilité des constantes de classe

Types nullable et type **void**

PHP 7.1

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark()
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
30 }
31 }
32 return $item;
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
30 }
31 }
32 return $item;
```


PHP 7.1

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

SOUND pourrait être de n'importe quel type

~~**SOUND** pourrait être lu depuis l'extérieur~~

\$name pourrait être de n'importe quel type

\$microchipId pourrait être de n'importe quel type ?

\$name et **\$microchipId** pourraient changer pendant l'exécution

~~**\$microchipId** accepte n'importe quel type dans le constructeur~~

fetch pourrait être appelée depuis l'extérieur

~~**bark** pourrait retourner n'importe quel type~~

PHP 7.4

Propriétés typées

PHP 7.4

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private $name;
10
11     private $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
30 }
```

```
31 }
```

```
32     return $item;
```

```
33     $this->bark();
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private string $name;
10
11     private ?int $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
30 }
```

```
31 }
```

```
32     return $item;
```

```
33     $this->bark();
```


PHP 7.4

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private string $name;
10
11     private ?int $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

SOUND pourrait être de n'importe quel type

~~**\$name** et **\$microchipId** pourraient être de n'importe quel type~~

\$name et **\$microchipId** pourraient changer pendant l'exécution

fetch pourrait être appelée depuis l'extérieur

PHP 8.0

Promotion de propriété de constructeur

PHP 8.0

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     private string $name;
10
11     private ?int $microchipId;
12
13     public function __construct(string $name, ?int $microchipId)
14     {
15         $this->name = $name;
16         $this->microchipId = $microchipId;
17     }
18
19     private function bark(): void
20     {
21         echo self::SOUND;
22     }
23
24     public function fetch(Ball $item): Ball
25     {
26         $this->bark();
27         return $item;
28     }
29 }
```

```
30 }
31
32 return $item;
33 $input2->bark();
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         private string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
35 }
36
37 return $item;
38 $input2->bark();
```


PHP 8.1

Propriétés en lecture seule (**readonly**)

PHP 8.1

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         private string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 return $item;
57 $this->bark();
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public readonly string $name,
11         private readonly ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 return $item;
57 $this->bark();
```


PHP 8.1

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public readonly string $name,
11         private readonly ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfnlu $ifgw:
57 $ifgw->bark():
```

SOUND pourrait être de n'importe quel type

~~**\$name** et **\$microchipId** pourraient changer pendant l'exécution~~

fetch pourrait être appelée depuis l'extérieur

PHP 8.1

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public readonly string $name,
11         private readonly ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
52         return $item;
51         $this->bark();
```

SOUND pourrait être de n'importe quel type

\$name pourrait être lue depuis l'extérieur

fetch pourrait être appelée depuis l'extérieur

PHP 8.2

Classes en lecture seule (**readonly**)

PHP 8.2

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public readonly string $name,
11         private readonly ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
52 return $item;
51 $this->bark();
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final readonly class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
52 return $item;
51 $this->bark();
```


PHP 8.3

Typage des constantes de classe

PHP 8.3

```
1 <?php
2
3 declare(strict_types=1);
4
5 final readonly class Dog
6 {
7     private const SOUND = 'Woof';
8
9     public function __construct(
10         public string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfnL $frew:
57 $frew->bark():
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final readonly class Dog
6 {
7     private const string SOUND = 'Woof';
8
9     public function __construct(
10         public string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfnL $frew:
57 $frew->bark():
```


PHP 8.3

```
1 <?php
2
3 declare(strict_types=1);
4
5 final readonly class Dog
6 {
7     private const string SOUND = 'Woof';
8
9     public function __construct(
10         public string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
24 }
23 }
22     let $item;
21     $item->bark();
```

~~SOUND~~ pourrait être de n'importe quel type

\$name pourrait être lue depuis l'extérieur

fetch pourrait être appelée depuis l'extérieur

PHP 8.4

Visibilité asymétrique

PHP 8.4

```
1 <?php
2
3 declare(strict_types=1);
4
5 final readonly class Dog
6 {
7     private const string SOUND = 'Woof';
8
9     public function __construct(
10         public string $name,
11         private ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfnL $frefw:
57 $frefw->bark():
```

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const string SOUND = 'Woof';
8
9     public function __construct(
10         public readonly string $name,
11         public private(set) ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
54 }
53 }
55 letfnL $frefw:
57 $frefw->bark():
```

PHP 8.4

```
1 <?php
2
3 declare(strict_types=1);
4
5 final class Dog
6 {
7     private const string SOUND = 'Woof';
8
9     public function __construct(
10         public readonly string $name,
11         public private(set) ?int $microchipId,
12     ) {}
13
14     private function bark(): void
15     {
16         echo self::SOUND;
17     }
18
19     public function fetch(Ball $item): Ball
20     {
21         $this->bark();
22         return $item;
23     }
24 }
```

```
24 }
23 }
22     return $item;
21     $this->bark();
```

\$name et **\$microchipId** pourraient être lues depuis l'extérieur

fetch pourrait être appelée depuis l'extérieur

Avant

Dog pourrait avoir des classes enfants

SOUND pourrait être de n'importe quel type

SOUND pourrait être lue depuis l'extérieur

\$name et **\$microchipld** pourraient être de n'importe quel type

\$name et **\$microchipld** pourraient être lues depuis l'extérieur

\$name et **\$microchipld** pourraient changer pendant l'exécution

\$name et **\$microchipld** pourraient être modifiées depuis l'extérieur

\$name et **\$microchipld** acceptent n'importe quel type dans le constructeur

bark et **fetch** pourraient être appelées depuis l'extérieur

bark et **fetch** pourraient retourner n'importe quel type

\$item accepte n'importe quel type dans la signature de **fetch**

Après

\$name et **\$microchipld** pourraient être lues depuis l'extérieur

fetch pourrait être appelée depuis l'extérieur

Autres fonctionnalités

PHP 5.3: résolution statique à la volée (**::static**)

PHP 7.1: type **iterable**

PHP 7.4: covariance / contravariance limitées

PHP 8.0: arguments nommés, types d'union, attributs

PHP 8.1: énumérations, types d'intersection, type de retour **never**, constantes de classe **final**

PHP 8.2: types sous forme normale disjonctive (DNF)

PHP 8.3: attribut **#[Override]**

PHP 8.4: hooks de propriétés, propriétés de classe **final**

PHP 8.5: propriétés promues **final**, attribut **#[NoDiscard]**

PHP 9.0: erreur fatale pour les propriétés dynamiques (sauf avec l'attribut **#[AllowDynamicProperties]**)

Analyse statique

Analyser le code sans l'exécuter

Analyse statique

Rector pour les refactorisations

<https://getrector.com>

PHP CS Fixer pour les règles de style de code

<https://cs.symfony.com>

PHPStan pour signaler les problèmes restants

<https://phpstan.org>

Rector

Type Declarations

Ajouter les types manquants (paramètres, propriétés, constantes, retours)

Rector\Set\ValueObject\SetList::TYPE_DECLARATION

<https://getrector.com/find-rule?rectorSet=core-type-declarations&activeRectorSetGroup=core>

Rector

ClassPropertyAssignToConstructorPromotionRector

<https://getrector.com/rule-detail/class-property-assign-to-constructor-promotion-rector>

Incluse dans les règles de **PHP 8.0** (**Rector\Set\ValueObject\LevelSetList::UP_TO_PHP_80**)

<https://getrector.com/find-rule?rectorSet=php-php-80&activeRectorSetGroup=php>

 Rector a des jeux de règles pour toutes les versions de PHP
Exemple : `Rector\Set\ValueObject\LevelSetList::UP_TO_PHP_85`

Rector

ReadOnlyPropertyRector

<https://getrector.com/rule-detail/read-only-property-rector>

Incluse dans les règles de **PHP 8.1** (**Rector\Set\ValueObject\LevelSetList::UP_TO_PHP_81**)

<https://getrector.com/find-rule?rectorSet=php-php-81&activeRectorSetGroup=php>

Rector

ReadOnlyClassRector (classes **final** uniquement)

<https://getrector.com/rule-detail/read-only-class-rector>

Incluse dans les règles de **PHP 8.2** (**Rector\Set\ValueObject\LevelSetList::UP_TO_PHP_82**)

<https://getrector.com/find-rule?rectorSet=php-php-82&activeRectorSetGroup=php>

Rector

SafeDeclareStrictTypesRector

<https://getrector.com/rule-detail/safe-declare-strict-types-rector>

Rector

Swiss Knife (**finalize-classes**)

<https://github.com/rectorphp/swiss-knife>

Bypass Finals (**DG\BypassFinals::enable();**)

<https://github.com/dg/bypass-finals>

PHP CS Fixer

modifier_keywords (incluse dans les règles **@PER-CS**)

https://cs.symfony.com/doc/rules/class_notation/modifier_keywords.html

protected_to_private

https://cs.symfony.com/doc/rules/class_notation/protected_to_private.html

PHPStan

Niveau 6

Signale les types manquants (paramètres, propriétés, retours)

<https://phpstan.org/user-guide/rule-levels>

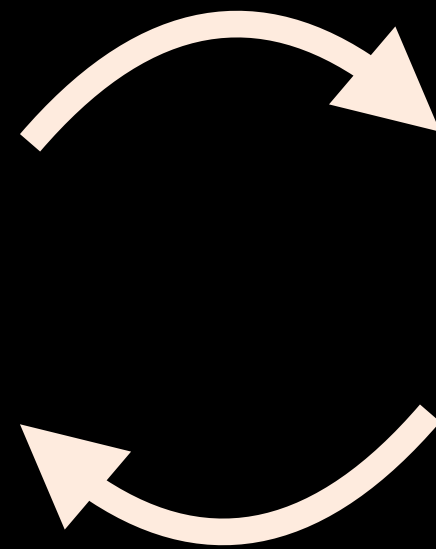
Type Coverage

Signale les types de constante manquants (**constant: 100**)

<https://github.com/TomasVotruba/type-coverage>



Code plus strict



Analyse statique plus performante

PHPStan

PHPDoc Types

<https://phpstan.org/writing-php-code/phpdoc-types>

Generics By Examples

<https://phpstan.org/blog/generics-by-examples>

Principes liés

Encapsulation

YAGNI

Immutabilité

KISS

Principe ouvert/fermé (Open/Closed)

Tell, Don't Ask

Principe de moindre privilège

Principe ouvert/fermé (Open/Closed)

◀◀ Une classe doit être à la fois ouverte à l'extension et fermée à la modification.

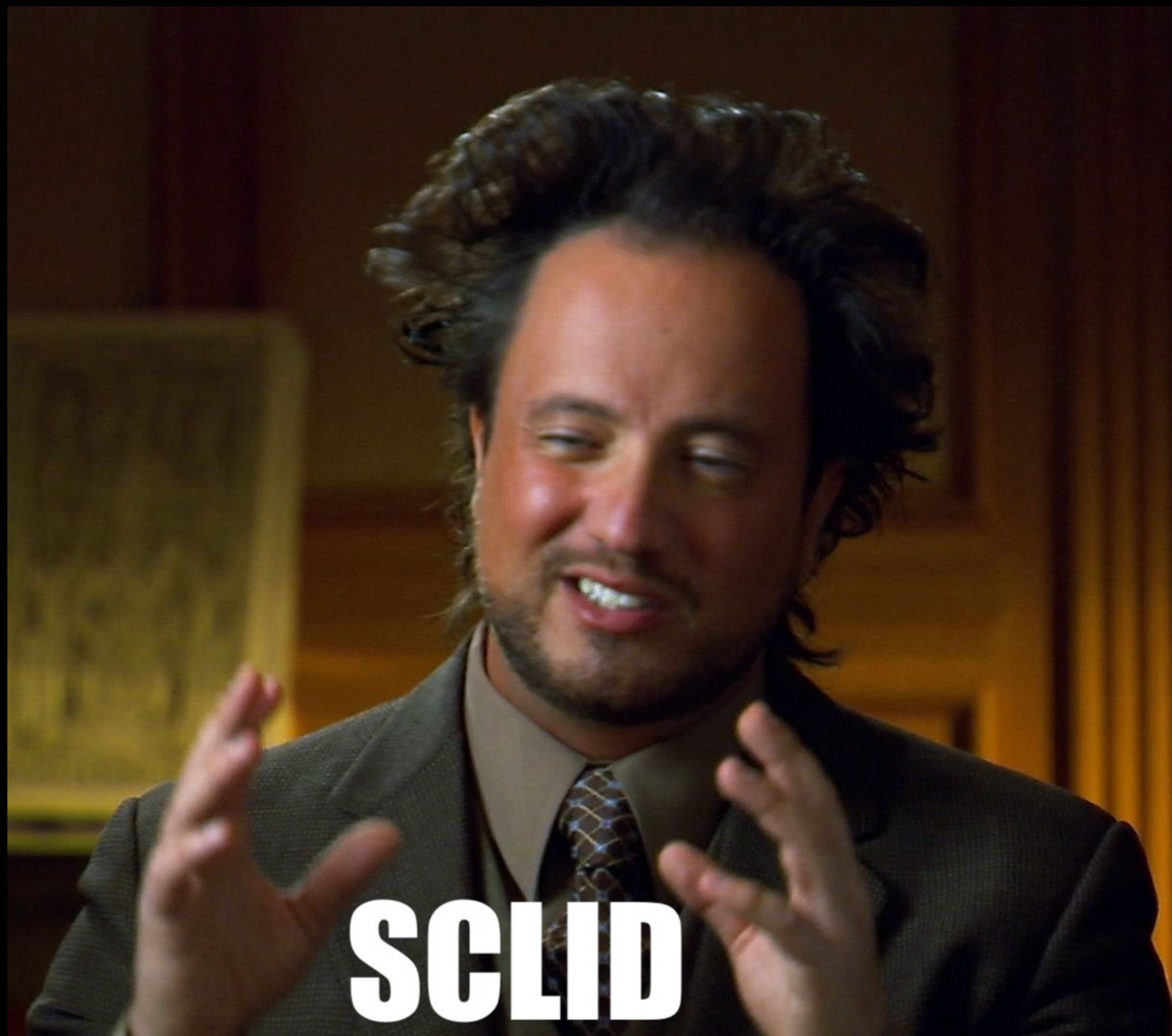
Principe Closed-by-Default

◀◀ Une classe ne devrait être ouverte à l'extension qu'en cas de besoin avéré.



Murmures chez les mainteneurs de packages...

Closed-by-Default ❤️ Open/Closed



Pour conclure...

Utilisez les constructions du langage et les patterns pour réduire la « surface cognitive » de votre code

Choisissez soigneusement où et quand vous ouvrez votre code

Utilisez l'analyse statique pour la cohérence et la prévisibilité

Pour conclure...

Value objects, **DateTimeImmutable**, constructeurs nommés...

Règles d'analyse statique personnalisées

Analyse statique de l'architecture (Deptrac, Pest... PHPStan)

Garde-fous pour le code généré par l'IA

« Le principe « Closed-by-Default » élimine les impasses de votre labyrinthe mental et révèle un chemin plus direct vers la compréhension du code.





Code et slides



Feedback

Merci !



Restons en contact